Hyperparameter experiments on Fashion MNIST

Vincent Brand | 1447806 | Machine Learning – MADS, HAN
2026 | https://github.com/vincentbrand/MADS-ML-Vincent

1 Hypothesis

Fashion-MNIST is a small (28×28, single-channel) 10-class problem, so I expect a dense network's performance to be limited mainly by the *width* of its hidden layers rather than its *depth*. My hypothesis is that on this simple data, widening the layers lowers test loss more than stacking extra layers, and that the *first* hidden layer — the one that sees the raw 784-pixel input — drives most of that gain. I also expect a clear learning-rate sweet spot and diminishing returns past a few epochs.

2 Experiment design

Every run uses the provided three-`Linear`-layer network (`CrossEntropyLoss`, Adam, `ReduceLROnPlateau`); I report the final test loss. From a 128×128 / lr$= 10^{-3}$ / batch$= 64$ / 5-epoch baseline I changed one factor at a time across six sweeps: a 4×4 width grid over units1$\times$units2 $\in \{32, 64, 128, 256\}$ (Fig. 1); depth (2/3/4 layers at 64/128/256 units); learning rate (10^{-2}–10^{-5}); optimizer (SGD/Adam/AdamW/RMSprop); batch size (8–128); and epochs (3/5/10) (Fig. 2). I used powers of two because they align with how memory and compute tiles are allocated on the hardware; the trade-off is a coarse grid that can step over a better value in between.

3 Results

As shown in Fig. 1, wider networks consistently reach lower test loss: the best two-layer model is 256×256 (0.334), the worst 32×32 (0.405). Averaged across the grid, widening the *first* layer from 32 to 256 units cuts loss by ≈ 0.040 but the *second* layer by only ≈ 0.018 — the first hidden layer matters about twice as much, as expected since it must compress the full 784-pixel input. Depth did not help: at 256 units the two-layer net (0.331) beat the three- and four-layer ones (0.339, 0.345), and at 128/64 units it was flat. The sweeps (Fig. 2) give a U-shaped learning-rate curve minimised near 10^{-3} (0.345): 10^{-2} overshoots (0.426) and 10^{-5} barely learns in 5 epochs (0.732). Adam, AdamW and RMSprop are interchangeable (≈ 0.34) and all crush SGD (1.21); batch size barely matters (0.35–0.39).

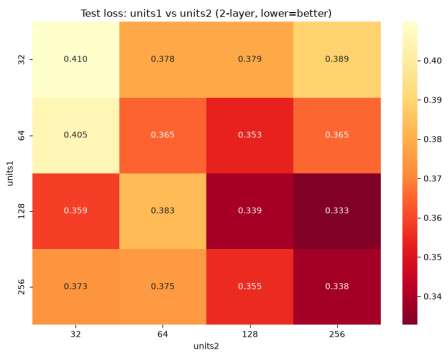

Figure 1: Test loss across units1 (first hidden layer) vs units2 (second), two-layer net; darker is better. The first layer's width has the larger effect.

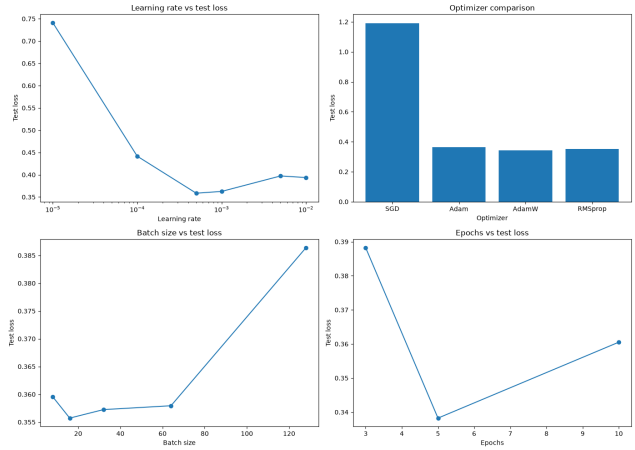

Figure 2: One-factor sweeps from the 128×128 baseline: learning rate (U-shaped, min near 10^{-3}), optimizer (SGD lags far behind the adaptive ones), batch size and epochs.

4 Conclusions

The results support my hypothesis: width beats depth here, and most of the width gain comes from the first hidden layer, because a shallow-but-wide MLP already has the capacity to separate the classes while extra layers mainly add optimisation difficulty without new structure the data can exploit. The epoch sweep shows both faces of longer training: $3 \rightarrow 5$ epochs helped ($0.375 \rightarrow 0.343$) but 10 was slightly worse (0.347) — an early sign of overfitting, so more epochs only pay off for harder data or smaller learning rates. Powers of two kept the search hardware-friendly, but the coarse grid likely skips the true optimum; the next step is a finer search around 256 units and lr$\approx 10^{-3}$ with a regulariser.